

EXERCISE SOLUTIONS

Redis Exercise Solutions

Model answers, working code and marking guidance

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this document

Model answers for the ten exercises, with the marking guidance under each one. Several have more than one defensible answer - the marking note says what must be present to earn the mark, not the exact words. The runnable versions of the five programs are in `node_cookbook/exercises/`.

Section	Exercises	Marks	Drawn from
redis-cli drills	1 - 4	21	Decks 3 and 5, labs 7.2 - 7.6
Node.js programs	5 - 9	32	Deck 5, labs 7.9 - 7.10, the cookbook
Operations	10	7	Deck 4, lab 7.8
Total		60	

FOR THE INSTRUCTOR Give credit for a correct answer that arrives by a different route - Redis usually has two or three reasonable ways to do the same thing, and recognising that is itself a sign the material landed. Do not give credit for code that was never run: every exercise asks for output, and the output is where the learning shows.

REDIS-CLI DRILLS

redis-cli drills

Exercise 1 [5 marks]

Build and inspect the balance cache

Everything below is typed at the `redis-cli` prompt. Write the command and the reply you actually got - not the reply you expected.

Model answer

- 1 - TTL on a write.

```
127.0.0.1:6379> SET icici:bal:9003 341244.80 EX 300
OK
127.0.0.1:6379> TTL icici:bal:9003
(integer) 300
```

- 2 - Discovery without blocking.

```
127.0.0.1:6379> SCAN 0 MATCH icici:bal:* COUNT 100
1) "0"
2) 1) "icici:bal:9001"
   2) "icici:bal:9002"
   3) "icici:bal:9003"
# loop until element 1 comes back "0"; from the shell:
$ redis-cli --scan --pattern 'icici:bal:*
```

- KEYS is O(n) over the whole keyspace and Redis runs one command at a time, so on a 20-million-key instance it stalls every other client for seconds.
- 3 - **Type and encoding.** TYPE → string, OBJECT ENCODING → embstr. A string of 44 bytes or fewer lives in a single allocation with its header - cheap to create, cache-friendly to read. Longer strings become raw.
- 4 - **Create-if-absent.**

```
127.0.0.1:6379> SET icici:bal:9004 5000 NX
OK
127.0.0.1:6379> SET icici:bal:9004 5000 NX
(nil)
127.0.0.1:6379> OBJECT ENCODING icici:bal:9004
"int"
```

- 5000 parses as a 64-bit integer so Redis stores the number, not nine bytes of text - which is what makes INCR possible with no parse. 341244.80 is not an integer, so it stays embstr.
- 5 - **Non-blocking invalidation.** UNLINK icici:bal:9003 → (integer) 1. DEL frees the memory on the command thread; UNLINK hands it to a background thread.

TIP On one small string the difference is nil. The habit matters the day the key is a two-million-member Set.

MARKING 1 mark per step. Step 2 needs SCAN and the reason. Step 4 needs both replies - one reply alone shows it was not run twice. Accept DEL in step 5 only if UNLINK is named as the better answer.

Exercise 2 [6 marks]

Pick the structure, then actually build it

Four requirements from the ICICI platform. For each: name the type, build it with the data given, and run a verification command that proves it works. A type name on its own scores nothing.

Model answer

- (a) List - push then trim. The trim is what makes it a cap.

```
$ for i in $(seq 1 12); do
>   redis-cli LPUSH icici:recent:9003 txn:$i &gt; /dev/null
>   redis-cli LTRIM icici:recent:9003 0 9 &gt; /dev/null
> done
$ redis-cli LLEN icici:recent:9003
(integer) 10
$ redis-cli LRANGE icici:recent:9003 0 2
1) "txn:12" 2) "txn:11" 3) "txn:10"
```

- (b) Sorted Set - ZINCRBY is read-modify-write on the server.

```
$ redis-cli ZADD icici:lb:branch:2026-09 250000 BR-MUM-014 410000 BR-DEL-002 \
185000 BR-BLR-031 322000 BR-CHN-009
(integer) 4
$ redis-cli ZINCRBY icici:lb:branch:2026-09 90000 BR-MUM-014
"340000"
$ redis-cli ZRANGE icici:lb:branch:2026-09 0 2 REV WITHSCORES
1) "BR-DEL-002" 2) "410000"
3) "BR-MUM-014" 4) "340000"
5) "BR-CHN-009" 6) "322000"
```

- (c) Set - the reply is the decision: 1 new, 0 duplicate.

```
$ redis-cli SADD icici:seen:req REQ-88213
(integer) 1 # first arrival - post the payment
$ redis-cli SADD icici:seen:req REQ-88213
(integer) 0 # duplicate - reject
$ redis-cli EXPIRE icici:seen:req 86400
(integer) 1
```

- A per-request key (SET icici:seen:REQ-88213 1 NX EX 86400) is equally correct and expires each id individually - say which trade-off you chose.
- (d) HyperLogLog vs Set - the memory number is the point.

```
$ for i in $(seq 1 100000); do echo "PFADD icici:uv:hll cust:$i"; done \
| redis-cli --pipe
$ for i in $(seq 1 100000); do echo "SADD icici:uv:set cust:$i"; done \
| redis-cli --pipe
$ redis-cli PFCOUNT icici:uv:hll
(integer) 100156 # true value 100000
$ redis-cli MEMORY USAGE icici:uv:hll
(integer) 12352 # ~12 KB, and it stays 12 KB at a billion ids
$ redis-cli MEMORY USAGE icici:uv:set
(integer) 6291560 # ~6 MB, and it grows linearly
# error = |100156-100000| / 100000 = 0.156% (spec: 0.81% std error)
```

- Exact numbers vary by version; any answer in the 12 KB vs several-MB range with an error well under 1% is correct.

MARKING 1 mark each for (a) and (c), 2 marks each for (b) and (d). Full marks need the verification output, not just the build commands. (b) loses a mark if the score is updated with ZADD after a ZSCORE read. (d) loses a mark without both memory numbers.

Exercise 3 [5 marks]

Re-model a JSON blob, and watch the encoding flip

The customer profile is one JSON string. Two servers update it at the same time and one update keeps disappearing. Re-model it, then look at what Redis does underneath.

Model answer

- 1 - One command.

```
$ redis-cli HSET icici:cust:9003 name "A. Sharma" tier gold logins 41 \
                                     branch BR-MUM-014 mobile 98xxxxxx21 kyc verified
(integer) 6
```

- 2 - Field-level access.

```
$ redis-cli HINCRBY icici:cust:9003 logins 1
(integer) 42
$ redis-cli HGET icici:cust:9003 tier
"gold"
```

- 3 - The failure it removes. The JSON version needs GET, parse, modify, serialise, SET. Two servers doing that concurrently lose one update - the second SET overwrites the first server's change wholesale. HINCRBY is one atomic server-side operation on one field, so there is nothing to lose.
- 4 - Encoding and cost. listpack (ziplist on 6.2 and earlier), about 176 bytes. A small Hash is not a hash table at all - it is a flat, contiguous block of field/value pairs, scanned linearly, which at six fields is faster and far smaller than a real hash table.
- 5 - Forcing the flip.

```
$ redis-cli CONFIG GET hash-max-listpack-entries
1) "hash-max-listpack-entries" 2) "128"
$ for i in $(seq 1 200); do echo "HSET icici:cust:9003 f$i v$i"; done \
| redis-cli --pipe
$ redis-cli OBJECT ENCODING icici:cust:9003
"hashtable"
$ redis-cli MEMORY USAGE icici:cust:9003
(integer) 15304
# also valid: CONFIG SET hash-max-listpack-entries 4, then HSET one field
```

- Past the threshold every field and value becomes its own allocation with its own header and pointers, so per-field overhead goes from a few bytes to roughly 50-70. The conversion is one-way - deleting fields does not convert it back.

IMPORTANT This is the biggest single lever on Redis memory. Many small Hashes below the threshold is the cheapest way to store millions of records; one giant Hash is the most expensive.

MARKING 1 mark per task. Task 2 needs HINCRBY, not HGET+HSET. Task 5 needs the command, the new encoding and a correct reason for the jump.

Exercise 4 [5 marks]

Debug the session store

This script is in production. Sessions should die after 30 minutes of inactivity; instead Redis memory has grown every day for a month. Save it as `sessions.sh`, run it, and fix it.

Model answer

- **1 - What you see.** 1800, then -1, -1, -1. A plain SET on an existing key replaces the value *and* discards its TTL, so the first request after login turns a 30-minute session into a permanent key.
- **2 - The one-word fix.**

```
redis-cli SET icici:sess:$SID "{...}" KEPTTL
# TTls now 1800, 1799, 1799, 1798 - a fixed 30-minute session
```

- **3 - Sliding window.** KEPTTL keeps the *original* deadline, which is not what was asked. Re-arm on every request:

```
redis-cli SET icici:sess:$SID "{...}" EX 1800      # re-arms each time
# or, when the value has not changed:
redis-cli EXPIRE icici:sess:$SID 1800 GT          # Redis 7+, never shortens

$ redis-cli SET icici:sess:8f3a '{}' EX 1800; sleep 5; redis-cli TTL icici:sess:8f3a
OK
(integer) 1795
$ redis-cli SET icici:sess:8f3a '{}' EX 1800; redis-cli TTL icici:sess:8f3a
OK
(integer) 1800      # clock reset - sliding window confirmed
```

TIP Worth knowing for the contrast: HSET on a Hash does *not* clear the key's TTL. Only the string-write commands do, and only when they replace the whole value.

- **4 - Safe counting.**

```
redis-cli --scan --pattern 'icici:sess:*' | wc -l
# --scan loops SCAN for you; O(COUNT) per call, never blocks
```

- **5 - Expiry telemetry.**

```
redis-cli INFO stats | grep expired_keys
expired_keys:41822
# flat expired_keys next to growing memory is this exact bug's signature
```

MARKING 1 mark per task. Task 1 needs the -1 observation, not just "the TTL is gone". Task 3 must be a re-arm (EX or EXPIRE GT) - repeating KEPTTL scores 0 because it does not slide.

NODE.JS PROGRAMS

Node.js programs

Exercise 5 [6 marks]

otp.js — OTP issue, verify and lock-out

Write the OTP service for the mobile app. An OTP is valid for 90 seconds, may be verified at most three times, and a fourth attempt locks the customer out for five minutes. It must be correct when the customer taps "verify" twice on two devices at the same instant.

Model answer

- Keys: `icici:otp:<mobile>` (the code), `icici:otp:tries:<mobile>` (the counter), `icici:otp:lock:<mobile>` (the lock-out).

```
const TTL = 90, MAX = 3, LOCK = 300;
const otpKey = (m) => `icici:otp:${m}`;
const tryKey = (m) => `icici:otp:tries:${m}`;
const lockKey = (m) => `icici:otp:lock:${m}`;

async function issue(client, mobile) {
  const code = String(Math.floor(100000 + Math.random() * 900000));
  await client.multi()
    .set(otpKey(mobile), code, { EX: TTL })
    .unlink(tryKey(mobile)) // a new OTP resets the attempts
    .exec();
  return code;
}

async function verify(client, mobile, code) {
  if (await client.exists(lockKey(mobile))) return 'LOCKED';

  const stored = await client.get(otpKey(mobile));
  if (stored === null) return 'EXPIRED';

  if (stored === code) {
    // Consume it. UNLINK returns 1 only for the caller that won,
    // so a simultaneous second tap cannot also get OK.
    const won = await client.unlink(otpKey(mobile));
    await client.unlink(tryKey(mobile));
    return won === 1 ? 'OK' : 'EXPIRED';
  }

  const [tries] = await client.multi()
    .incr(tryKey(mobile))
    .expire(tryKey(mobile), TTL, 'NX') // arm once, never extend
    .exec();

  if (tries >= MAX) {
    await client.multi()
      .set(lockKey(mobile), '1', { EX: LOCK })
      .unlink(otpKey(mobile))
      .exec();
    return 'LOCKED';
  }
  return 'WRONG';
}
```

- **Expected output.**

```
issued 412907
verify 000000 → WRONG (attempt 1 of 3)
verify 412907 → OK
verify 412907 → EXPIRED (replay: the OTP was consumed)
verify 111111 → WRONG · WRONG · LOCKED
TTL icici:otp:lock:9876543210 → 300
```

- **The three things being marked.** EXPIRE . . . NX so a later wrong attempt cannot extend the attempt window; UNLINK's 1/0 reply to consume the OTP exactly once under concurrency; and a TTL on every one of the three keys so nothing is left behind.

IMPORTANT Reading the counter with GET and then deciding in JavaScript is the classic wrong answer - two devices both read 2, both decide "not locked yet", and the customer gets four attempts. Let INCR return the number you branch on.

MARKING 1 mark per task (6). Deduct 1 if the attempt counter has no TTL, or if it is re-armed on every attempt. Deduct 1 if a replay of a correct code returns OK twice. Accept a Lua implementation - it is a better answer, not a worse one.

Exercise 6 [7 marks]

cache.js — cache-aside, and its two failure modes

You are given a stub "core banking" module whose every read costs 50 ms. Put Redis in front of it, then reproduce and fix the two bugs every cache eventually has.

Model answer

- 1, 3, 5 - the pattern, with all four rules in it.

```
async function getBalance(client, acct) {
  const k = `icici:bal:${acct}`;
  try {
    // RULE 4 - fail open
    const hit = await client.get(k);
    if (hit !== null) return { balance: Number(hit), source: 'HIT' };
  } catch { /* Redis is down - fall through to the database */ }

  const balance = await core.selectBalance(acct);
  if (balance === null) return { balance: null, source: 'MISS' };

  try {
    const jitter = Math.floor(Math.random() * 60); // RULE 1, and bug 2
    await client.set(k, String(balance), { EX: 300 + jitter });
  } catch { /* ignore */ }
  return { balance, source: 'MISS' };
}

async function postDebit(client, acct, amount) {
  const balance = await core.applyDebit(acct, amount);
  await client.unlink(`icici:bal:${acct}`); // RULES 2 and 3
  return balance;
}
```

- **Bug 1 - stale reads.** The write path updates the core system but nothing removes the cached balance, so the old figure is served until the TTL expires. The customer sees money they have already spent. Fix: UNLINK in the same code path as the debit. Delete rather than overwrite - an overwrite can race with a read already on its way back with the old value.
- **Bug 2 - cache stampede.** Every key was populated in the same loop with the same TTL, so they all expire on the same second; every request misses at once and the whole day's traffic arrives at the database simultaneously. Fix: jitter the TTL, as above. For genuinely hot keys also guard the refill:

```
const gotLock = await client.set(`lock:refill:${k}`, token,
  { NX: true, PX: 5000 });
// only the winner queries the database; everyone else waits or
// serves the last known value
```

- **Expected output.**

```
MISS balance 341244.8 in 51.44 ms
HIT average 0.180 ms speed-up 286x
after debit, no UNLINK: cached 341244.8 actual 340000.0 ← stale
after debit, with UNLINK: (nil) → next read repopulates → 340000.0
redis down: MISS 340000.0 in 51.02 ms (degraded, not broken)
hit rate 99.3%
```

- **6 - the metric.** $\text{keyspace_hits} / (\text{keyspace_hits} + \text{keyspace_misses})$ from INFO stats. Below roughly 80% the TTL or the key design needs rethinking; p99 latency on the API is the supporting metric.

WATCH OUT The fail-open test is the one candidates skip. An API that returns 500 because the *cache* is down has made availability worse, not better.

MARKING 2 marks for a working cache-aside with a TTL and the measurement. 2 marks for bug 1 demonstrated and fixed with UNLINK in the write path. 2 marks for bug 2 named as a stampede and fixed with jitter. 1 mark for fail-open plus the hit rate. Deduct 1 for SET-ing the new value instead of deleting it.

Exercise 7 [6 marks]

lock.js — only one server runs the batch

End-of-day reconciliation runs at 23:30 on six app servers. Exactly one must run it. Twice would double-post every entry.

Model answer

- 1, 2, 3 - the lock.

```
import { randomUUID } from 'node:crypto';

const RELEASE = `
  if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("DEL", KEYS[1])
  else
    return 0
  end`;

const EXTEND = `
  if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("PEXPIRE", KEYS[1], ARGV[2])
  else return 0 end`;

async function acquire(client, ttlMs = 30000) {
  const token = randomUUID();
  const ok = await client.set(LOCK, token, { NX: true, PX: ttlMs });
  return ok ? token : null;
}

const release = (client, token) => {
  client.eval(RELEASE, { keys: [LOCK], arguments: [token] });
}
```

- **Why each part is there.** NX - only one caller can create the key, so only one server wins. PX - the lock expires by itself; without it a server that crashes holding the lock blocks the batch permanently. The random token - the release checks ownership.
- **Why release cannot be two commands.** Between GET and DEL the lease can expire and another server can take the lock; the DEL then deletes *their* lock and two batches run. A Lua script is one command and Redis runs it to completion with nothing interleaved.
- **Expected output.**

```
app-01 HOLDS the lock, token 7f3c9a11... - running reconciliation
app-02 did not get the lock - backing off (9847 ms left on it)
app-03 did not get the lock - backing off (9846 ms left on it)
app-01 finished, released the lock (script returned 1)
servers that ran the batch: 1
release with a foreign token → 0   refused
release with the right token → 1   deleted
```

- **6 - the remaining failure.** On a replicated Redis a failover can promote a replica that never received the lock write, and the lock is then handed to a second server. If that would be a financial incident, use Redlock across independent masters, or keep the guarantee in PostgreSQL where the transaction already is.

TIP A watchdog that calls extend() every TTL/3 is how a long batch stays safe without setting an enormous lease that would strand the lock after a crash.

MARKING 1 mark each for acquire, release-in-Lua, extend, the three-worker proof, the wrong-token proof, and the replication caveat. A release written as GET-then-DEL scores 0 for that task however well explained.

Exercise 8 [6 marks]

queue.js — a job queue that does not lose work

"Email me my statement" queues a PDF job consumed by several workers. Build the queue, then kill a worker mid-job and prove nothing was lost.

Model answer

- **The two-list pattern.** The job moves; it is never deleted until it is done.

```
const QUEUE = 'icici:q:statements';
const processing = (w) => `${QUEUE}:processing:${w}`;

// producer
await client.lPush(QUEUE, JSON.stringify(job));

// claim - atomic move, blocks up to 2s instead of polling
const raw = await client.blMove(QUEUE, processing(w), 'RIGHT', 'LEFT', 2);
if (raw === null) return; // timed out: queue drained

await generatePdf(JSON.parse(raw));

// acknowledge - only now does the job stop existing
await client.lRem(processing(w), 1, raw);

// the janitor
async function requeueStale(client, w) {
  let n = 0;
  while (await client.lMove(processing(w), QUEUE, 'RIGHT', 'LEFT')) n++;
  return n;
}
```

- **Expected output.**

```
w1 completed 9: stmt-10, stmt-9, stmt-8, stmt-7, ...
w2 picked stmt-4 and crashed before acknowledging it
LLEN icici:q:statements → 0           the queue is empty
LRANGE ...:processing:w2 0 -1 → ["stmt-4"] ← still there, recoverable
requeueStale('w2') → 1 job requeued
w3 completed 1: stmt-4           LLEN → 0   nothing in flight
```

- **6 - why BRPOP cannot.** BRPOP removes the job from the list at the moment it hands it to the client. Crash one line later and the job does not exist anywhere - not in the queue, not in a processing list, not in a log. There is nothing to recover because there is no record that it was ever taken.

TIP LPUSH on the head and BLMOVE from the tail is what makes it FIFO. Push and pop from the same end and you have built a stack, which for a job queue means the oldest job is served last - or never, under load.

- **In production, use Streams.** A consumer group does all of this bookkeeping inside Redis - XREADGROUP delivers, XACK confirms, XPENDING lists what is outstanding and XAUTOCLAIM takes over a dead consumer's backlog. Build this once to understand the problem, then use recipe 12.

MARKING 1 mark per task. Task 2 scores 0 if claim and acknowledge are one command. Task 4 must show the job in the processing list, not merely assert it. Accept a Streams-based answer for full marks if the crash and recovery are demonstrated.

Exercise 9 [7 marks]

dashboard.js — four structures, one report

The regional head wants one screen, refreshed every ten seconds, from live data. Each figure needs a different structure - choosing correctly is most of the marks.

Model answer

- The four structures, one per question.

```
// (a) top 5 + rank → Sorted Set
const top = await client.zRangeWithScores(BOARD, 0, 4, { REV: true });
const rank = await client.zRevRank(BOARD, 'BR-CHN-009'); // 0-based
const score = await client.zScore(BOARD, 'BR-CHN-009');

// (b) unique customers → HyperLogLog
const today = await client.pfCount(`icici:uv:${day}`);
const week = await client.pfCount(last7Keys); // unions, no double-count

// (c) active on all 7 days → Bitmap
await client.bitOp('AND', 'icici:dau:sticky', last7DauKeys);
const retained = await client.bitCount('icici:dau:sticky');

// (d) what it costs
let bytes = 0;
for (const k of everyKey) bytes += await client.memoryUsage(k);
```

- **Why each one.** Ranking and "where am I" are $O(\log n)$ on a Sorted Set and would be an ORDER BY over the transaction table anywhere else. Distinct counts are a HyperLogLog because nobody needs them exact and a Set of 50,000 ids costs 3 MB a day. "Active on all seven days" is a Bitmap because it is bit arithmetic - BITOP AND over seven keys, then one BITCOUNT.

- Expected report.

```
ICICI · WEST REGION · 2026-09-22 18:00
```

```
TOP BRANCHES          MTD VOLUME
```

1. BR-DEL-002	■4,10,000
2. BR-MUM-014	■3,40,000
3. BR-CHN-009	■3,22,000
4. BR-BLR-031	■1,85,000
5. BR-PNQ-007	■96,500

BR-CHN-009 is #3 of 20

unique customers today	18,432	
unique customers this week	47,915	(not the sum of the days)
active every day this week	1,666	
dashboard keyspace	148 KB	

- **7 - the scale argument.** The leaderboard is one key of 20 members; the HyperLogLogs are 12 KB per day whatever the traffic; the bitmaps are one bit per customer per day, so 50 million customers cost 6 MB a day. The whole dashboard is kilobytes and its cost does not grow with transaction volume - only with the number of customers and days retained.

WATCH OUT The wrong answers to watch for: a Set for the unique counts (correct, but hundreds of times the memory, and the exercise says "roughly"); summing the seven daily figures for the weekly one (double-counts every returning customer); and ZRANGE of all 20 branches followed by indexOf in JavaScript (works at 20 branches, not at 4,000).

MARKING 2 marks for the leaderboard and the rank, 2 for both HLL figures with PFCOUNT over several keys or PFMERGE, 2 for the BITOP AND retention figure, 1 for the memory total and the scale argument. Deduct 2 for summing daily uniques.

OPERATIONS

Operations

Exercise 10 [7 marks]

Break it, then write the health check

Do this on a scratch instance on port 6380, never on a shared one. You are going to run Redis out of memory on purpose, watch what it does, and then write the script an on-call engineer would run at 02:00.

Model answer

- 1 - Breaking it.

```
$ redis-benchmark -p 6380 -t set -n 200000 -r 200000 -d 200 -q
OOM command not allowed when used memory > 'maxmemory'.
# maxmemory-policy noeviction - the default. Redis refuses writes
# rather than deleting anything. Reads and DELs still work.
```

- 2 - The fix for a cache.

```
$ redis-cli -p 6380 CONFIG SET maxmemory-policy allkeys-lru
$ redis-cli -p 6380 CONFIG SET maxmemory 64mb
$ redis-cli -p 6380 CONFIG REWRITE # or it is lost on restart
```

- Size maxmemory at roughly 70% of the box, leaving headroom for the fork that BGSAVE performs.

- 3 - Proof of eviction.

```
$ redis-cli -p 6380 INFO stats | grep evicted_keys
evicted_keys:38417 # was 0 before the policy change
```

- 4 - **Memory health.** mem_fragmentation_ratio is RSS divided by the memory Redis asked for. 1.6 means 60% more resident memory than data - the allocator is holding pages it cannot return, usually after a large delete. Below 1.0 is worse: the OS is swapping, and Redis on swap is not a cache any more. Activedefrag or a restart during a maintenance window are the remedies.

- 5 - **Why not on a queue.** allkeys-lru will silently evict queued jobs. That is work which never happens and leaves no trace - no error, no dead letter, nothing. A queue instance keeps noeviction, is sized for its peak depth, and is alerted on well before it fills.

- 6, 7 - the health check.

```
const info = Object.fromEntries(
  (await client.info()).split('\n')
    .filter((l) => l.includes(':') && !l.startsWith('#'))
    .map((l) => l.trim().split(':')));

const hits = +info.keyspace_hits, misses = +info.keyspace_misses;
const hitRate = (100 * hits) / Math.max(1, hits + misses);
const frag = +info.mem_fragmentation_ratio;
const usedPct = +info.maxmemory ? (100 * info.used_memory) / info.maxmemory : 0;

const checks = [
  ['rdb_last_bgsave_status', info.rdb_last_bgsave_status === 'ok', 'CRIT'],
  ['memory headroom', usedPct < 85, 'WARN'],
  ['hit rate', hitRate > 80, 'WARN'],
  ['fragmentation', frag < 1.5, 'WARN'],
];

const failed = checks.filter(([, ok]) => !ok);
```

```
process.exitCode = failed.some(([, , sev]) => sev === 'CRIT') ? 2
  : failed.length ? 1 : 0;

// node-redis has no slowlogGet(); sendCommand takes any command
const slow = await client.sendCommand(['SLOWLOG', 'GET', '3']);
for (const [, , micros, args] of slow)
  console.log(micros + ' µs ' + args.join(' '));
```

- **Expected output.**

```
redis 7.2.4 up 3d 04:11 clients 42
memory      812.4M / 1.00G (79%) frag 1.08
hit rate    95.3% evicted 0   expired 41,822
last save   ok (114s ago)
WARN memory headroom → exit 1
```

CHECKPOINT The exit code is the part that matters. A health script that prints beautifully and always exits 0 cannot be wired into anything.

MARKING 1 mark each for the exact OOM text, the two CONFIG SETs with CONFIG REWRITE, the evicted_keys proof, the fragmentation reading with an interpretation, and the queue caveat. 2 marks for health.js with correct exit codes. Deduct 1 if CONFIG REWRITE is missing.

If the score was below 42

Map each exercise that went badly back to its source and re-run it. These answers are muscle memory, not recall - reading the solution a second time does very little.

Exercise	Revise	Then re-run
1	Deck 5 - key lifecycle, SCAN, encodings	Lab 7.2
2	Deck 3 - the nine types and what they cost	Labs 7.5 - 7.6
3	Deck 3 - Hash encodings and memory	Lab 7.4
4	Deck 5 - SET options, TTL semantics	Lab 7.3
5	Deck 5 - atomic counters and expiry	Cookbook 01, 08
6	Deck 2 - cache-aside and its failure modes	Lab 7.9 · cookbook 02
7	Deck 5 - transactions, Lua, locking	Lab 7.10 · cookbook 14
8	Deck 5 - lists, blocking commands, streams	Cookbook 05, 12
9	Deck 3 - choosing a structure	Cookbook 07, 09, 10
10	Deck 4 - maxmemory, eviction, persistence	Lab 7.8

NOTE The five programs in this document are also in `node_cookbook/exercises/` as files that run. Run the solution against your own Redis and compare its output with yours before deciding you got it wrong - some differences are just timing and memory, and those numbers are supposed to differ.